



LeetCode 876 — Middle of the Linked List

 <https://leetcode.com/problems/middle-of-the-linked-list/>

1. Problem Title & Link

Title: LeetCode 876: Middle of the Linked List

Link: <https://leetcode.com/problems/middle-of-the-linked-list/>

2. Problem Statement (Simple Summary)

Given the head of a singly linked list, return the **middle node**.

If two middle nodes exist (even length), return the **second middle node**.

Example:

1 → 2 → 3 → 4 → 5 → return 3

1 → 2 → 3 → 4 → 5 → 6 → return 4 (2nd middle)

3. Examples (Input → Output)

Example 1

Input: 1 → 2 → 3 → 4 → 5

Output: 3

Example 2

Input: 1 → 2 → 3 → 4 → 5 → 6

Output: 4

4. Constraints

- $1 \leq \text{length} \leq 100$
- Constant space expected
- $O(n)$ or $O(n/2)$ acceptable

5. Core Concept / Topic

Fast & Slow Pointer (Tortoise & Hare)

6. Thought Process — Step-by-Step

Approach 1 — Two Pointers (BEST)

- Slow moves 1 step
- Fast moves 2 steps
- When fast reaches end → slow is at middle
- For even-length list, slow reaches 2nd middle automatically

Approach 2 — Count + second traversal



- Count length $\rightarrow n$
- Move $n//2$ steps $\rightarrow$ middle
- But requires two passes $\rightarrow$ less optimal

7. Visual / Intuition Diagram

```
For list:
1 → 2 → 3 → 4 → 5
S      S      S      S(end)
F              F(end)
Middle → 3
Even case:
1 → 2 → 3 → 4 → 5 → 6
Slow ends on 4 ← correct
```

8. Pseudocode

```
slow = head
fast = head

while fast != null AND fast.next != null:
    slow = slow.next
    fast = fast.next.next

return slow    # slow is middle
```

9. Code Implementation

Python

```
class Solution:
    def middleNode(self, head):
        slow = fast = head
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        return slow
```

Java

```
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
        }
        return slow;
    }
}
```



```

        fast = fast.next.next;
    }
    return slow;
}
}

```

10. Time & Space Complexity

Complexity	Value
Time	O(n) (single pass)
Space	O(1) (no extra memory)

11. Common Mistakes / Edge Cases

- ✗ Using fast pointer wrong (must check fast & fast.next)
- ✗ Returning slow too early
- ✗ Counting approach but returning wrong index
- ✓ Handles even/odd lengths naturally
- ✓ Single node list returns itself

12. Dry Run (Example)

For 1 2 3 4 5 6:

Step	slow	fast
start	1	1
1	2	3
2	3	5
3	4	null → stop

Output → 4 ✓

13. Use Cases

- Central node detection
- Splitting linked list in half
- Palindrome linked list using slow pointer
- Cycle detection variant (Floyd's Algo)

14. Common Traps



- ⚠ Missing fast.next check → null pointer crash
- ⚠ Returning wrong node for even length
- ⚠ Looping until fast becomes null incorrectly

15. Builds To (Related Problems)

Problem	Concept
LC 141 — Cycle Detection	Fast & slow pointer
LC 143 — Reorder List	Middle + reverse
LC 234 — Palindrome Linked List	Middle + reverse
LC 19 — Remove Nth Node	Two-pointer technique

16. Alternate Approaches

Approach	Time	Space	Notes
Fast-Slow Pointer	$O(n)$	$O(1)$	Best
Two pass count	$O(n)$	$O(1)$	Slower but simple
Array conversion	$O(n)$	$O(n)$	Easy but extra memory

17. Why This Works

Fast moves twice as fast, so when fast hits the end, slow has traveled **half the length** → directly points to the middle.

18. Follow-Up

- Return **first** middle instead of second? (small change)
- Find left-middle or right-middle intentionally
- Middle in circular linked list?
- Middle with stream input? (slow pointer still applies)